DAYANANDA SAGAR UNIVERSITY

AI IN SOFTWARE ENGINEERING

CASE 57

Airline Crew Scheduling

Practical Work, Theory Answers and Laboratory Evidence

STUDENT / CASE DETAIL	VALUE
Student	Manoj
Roll number	57
Programme	B.Tech Computer Science and Engineering
University	Dayananda Sagar University
Course	AI in Software Engineering
Academic year	2026–2027
Class / section	[Not provided]
Submission date	[Not provided]
Process model	Spiral
Agile method	Scrum
AI tool specified	ChatGPT
Prototype	Duty-Limit Validator

GitHub repository URL (confirmed by the student):
https://github.com/manojshet2005/Airline-Crew-Scheduling.
Verified Git remote: https://github.com/manojshet2005/Airline-Crew-Scheduling..git (feature/weekly-limit).

Academic report prepared using the official Case 57 assignment and the project evidence available on 11 November 2026.

Table of Contents

SECTION	CONTENTS
Project context and evidence convention	Scope, sample-rule assumptions, status definitions
Part 1 – Case Tasks	Tasks A–E: Spiral, Scrum, prototype, testing, quality
Part 2 – Theory	T1.1–T1.5 | SDLC and process models
Part 2 – Theory	T2.1–T2.5 | Agile, Scrum, planning
Part 2 – Theory	T3.1–T3.5 | AI tools, Git, CI/CD
Part 2 – Theory	T4.1–T4.5 | Testing and defect prediction
Part 2 – Theory	T5.1–T5.5 | Quality, security, ethics
Part 3 – Laboratory Exercises	L1.1–L1.5 | SDLC and AI tools
Part 3 – Laboratory Exercises	L2.1–L2.5 | Agile and planning
Part 3 – Laboratory Exercises	L3.1–L3.5 | Prompting, Git, Jenkins, Compose
Part 3 – Laboratory Exercises	L4.1–L4.5 | Testing and monitoring
Part 3 – Laboratory Exercises	L5.1–L5.5 | Quality and responsible AI
References	Official assignment and project sources
Appendix A – AI Usage Log	Tools, purpose, verification, team policy
Appendix B – Disclosure and Evidence	Disclosure, eight genuine figures, evidence inventory

Project Context and Evidence Convention

The assignment scenario is an airline scheduling 400 crew members under duty-time laws, where a violation can ground a flight. The required choices are fixed: Spiral process model, Scrum agile method, ChatGPT, and a duty-limit validator. This submission reports the actual small Python prototype and the evidence audit; it does not claim a production airline roster system.

The implemented validator uses sample assumptions: daily duty at most 10 hours, minimum rest 10 hours, rolling seven-day duty at most 60 hours, and negative inputs raise ValueError. These values are not confirmed aviation law. A Flask dashboard now sends numeric duty totals to a JSON endpoint that calls the same validator and rejects invalid/non-finite request values. The prototype still does not compute duty from timestamps, produce a roster, persist crew data, or demonstrate 400-person performance.

The live UI was tested in Chrome at desktop and 390-pixel mobile widths. The browser checks covered acceptance, each of the three exact rule violations, reset, and missing-field feedback. Figures 4–5 in Appendix B are genuine screenshots from that run. This UI work does not change the existing lab status totals or imply that earlier L4.3 fixture screenshots were captures of this application.

Status	Meaning in this report
COMPLETE	The documented lab deliverable was performed and supported by its recorded evidence.
PARTIAL	Some work exists, but an explicitly required tool, evidence item, comparison, or execution is missing.
PENDING HUMAN	Genuine classmate/client participation or sign-off is still required.
BLOCKED	Required infrastructure or tool was unavailable; prepared configuration is not treated as execution.

Part 1 – Case Tasks

All five task answers follow the official Assignment Book, Case 57, printed page 228. The prototype scope remains intentionally narrow.

Task A – Module 1: Spiral lifecycle and AI risk analysis

The Spiral model fits this case because duty-rule ambiguity can create high-impact errors. Each iteration sets objectives, identifies and mitigates risks, develops and tests an increment, then reviews the result and plans the next loop. The first proposed loop makes the rule table and validator behavior explicit; later loops should add integration only after an authorized rule owner confirms the rules.

Risk to resolve	Why it matters	Control / evidence
Definition of a duty day across midnight	A different boundary can change daily duty totals.	Ask the rule owner; do not infer from a prompt.
Standby treatment	Counting or excluding standby changes duty totals.	Record as unresolved; obtain authoritative interpretation.
Daily/rest/weekly interactions	A schedule can satisfy one limit and violate another.	Define precedence and test exact boundaries and combined violations.
AI-invented or incomplete requirements	An unsupported rule could reject or accept a schedule incorrectly.	Trace to source, label vague/invented items, and review with a human.
Input edge cases	Wrong types or non-finite numbers can bypass intended checks.	Add explicit type/finite checks and tests before production use.

AI can assist with alternative wording, risk questions, and candidate tests. Rule approval, risk acceptance, and release decisions remain human-led. No formal operational Spiral cycle is claimed as executed; the model is the required process choice and the project evidence records planning and testing artifacts.

Task B – Module 2: Scrum and AI-assisted constraint planning

The backlog orders the first increment around the three sample constraints and an explainable rejection reason. US-01 (daily limit), US-02 (rest), US-03 (rolling seven-day limit), and US-04 (rule-specific rejection) are Must items. Input hardening is a Should; decision logging and roster import are Could; roster fairness reporting is explicitly outside the first validator prototype.

A sample two-week sprint goal is: demonstrate a Python validator that checks daily, minimum-rest, and rolling-week sample rules, rejects negative values, and explains each result. The Definition of Done includes documented assumptions, inclusive boundaries, automated tests, a green local suite, and no claim that the thresholds are law. L2.1–L2.3 planning artifacts are documented. L2.4 retains the actual ChatGPT estimates and a separate, explicitly AI-generated three-role simulation; actual classmate estimates and the real team comparison remain pending.

AI may help draft candidate stories and test cases, but story-point estimates are not team consensus. The evidence records both useful AI-generated structure and failures/incompleteness, including L3.1 Prompt 4 and the unsupported Boolean expectation in L1.5.

Task C – Module 3: Validator, Git, Jenkins, and Docker

The current implementation is a focused function in duty_validator.py. It checks negative values, then daily duty, minimum rest, and weekly duty; it returns a boolean and a reason, or raises ValueError for negative input. The limit values are assumptions, not confirmed law.

```
def validate_duty(hours_today, hours_last_7_days, rest_hours):
for hours in (hours_today, hours_last_7_days, rest_hours):
if hours < 0:
raise ValueError("Duty hours and rest hours cannot be negative.")
if hours_today > 10:
return False, "Daily duty limit exceeded."
if rest_hours < 10:
return False, "Minimum rest requirement not met."
if hours_last_7_days > 60:
return False, "Weekly duty limit exceeded."
return True, "Duty schedule accepted."
```

The original validator test files contain 20 tests; the expanded current local suite, including the Flask UI/API tests, produced 36 passed. The dashboard and API were exercised in real Chrome. Jenkins build #6 actually checked out the exact two-dot GitHub remote, installed dependencies, ran its separate remote-branch suite (9 passed), completed pip check, packaged the source archive, and finished SUCCESS. The required deliberate failing-test then corrected-success Jenkins demonstration remains outstanding. A genuine Docker Compose run built the validator image, connected it to healthy

PostgreSQL, and verified persisted rows; the validator was a one-shot service, not a persistent production service.

A real Git merge-conflict exercise was performed in an isolated temporary repository and documented; it did not alter the project branch history. The submitted project state is maintained on feature/weekly-limit; no force-push or history rewrite is part of this work.

Task D – Module 4: Testing, AI test cases, and defect prediction

The two project test files contain ten tests each. The requested full run passed all 20. Separate direct invocations documented ten manual checks and two deliberately seeded boundary defects in temporary functions. The AI-generated test review found 9/10 test designs supported by the Case 57 rule table (90%); the Boolean rejection expectation was not supported by the rule table and was removed from the corrected set. This does not prove legal correctness.

Both plain Selenium and Page Object Model scripts executed with Chrome against a local HTML fixture; boundary and daily-violation assertions passed. The POM reduces the locator update from three edits to one. The monitoring dashboard is a design sketch with mock values only. No 400-crew timing data or model-quality dataset was collected; defect prediction remains a proposed risk-prioritization aid, not a measured result.

Task E – Module 5: Quality, SonarQube, and fairness

The current validator returns a rule-specific reason and does not use a name or demographic attribute. That narrow property is not evidence that a future roster generator would be fair. A real roster feature would require defined fairness criteria, representative evaluation, appeal/review paths, and accountable human oversight.

Pylint, Flake8, and Bandit records are distinct from SonarQube. L5.2 documents Pylint findings without substituting them for SonarQube; no SonarQube scan or quality gate was run. L5.3 records a local AI-assisted review of the actual merged PR #7 diff; GitHub has zero posted review comments, and this is not represented as a posted review or human approval. L5.4's project implementation passed 10/10 shared cases and the AI reference passed 8/10; three runtime microbenchmark runs are recorded, but authoring-time and SonarQube evidence remain missing. L5.1 includes a separate AI simulation, while genuine peer review still requires a classmate.

Part 2 – Theory Questions

Question IDs and wording below follow the official Case 57 assignment. Each answer is written for the Airline Crew Scheduling scenario and separates documented facts from design proposals.

Module 1 – Software Engineering, SDLC and AI Tools

T1.1

Question: [Basic] List and explain the phases of the Software Development Life Cycle. For Airline Crew Scheduling, state one deliverable of each phase.

The SDLC gives a disciplined sequence for planning, specifying, designing, building, verifying, releasing, and maintaining software. For this case, the sequence is iterative: test feedback can return to requirements or design.

Phase	Case deliverable
Planning	Scope, stakeholders, 400-crew scenario, risks, and prototype milestone.
Requirements	Approved duty-rule source, input/output contract, and acceptance criteria.
Design	Validator flow, rule precedence, interface decision, and data assumptions.
Implementation	duty_validator.py and any supporting UI/API only if approved.
Testing	Boundary, violation, invalid-input tests and results.
Deployment	Reproducible package and instructions; not a production deployment.
Maintenance	Defect log and controlled updates when approved rules change.

T1.2

Question: [Basic] Explain the Spiral model with a neat diagram. Give two reasons it fits Airline Crew Scheduling and two situations in which you would change to another model.

A Spiral iteration repeats four activities: set objectives and alternatives; identify and resolve risks; develop and test an increment; review with stakeholders and plan the next loop. Risk analysis is central, and each cycle may produce a more complete prototype.

Diagram (text rendering):

```
Case loop: Objectives / alternatives → Risk analysis and mitigation → Develop and test a
small increment → Stakeholder review and plan next loop → (repeat).
```

- Fit reason 1: rule definitions and overnight/standby interpretation have material uncertainty, so risks can be addressed before broad implementation.
- Fit reason 2: each cycle can deliver a testable increment and collect stakeholder feedback on boundaries and explanations.
- Change to an incremental model if the rules are stable and the main objective is staged feature delivery; change to Waterfall if a regulator/customer requires a fixed, fully approved specification and gated documentation sequence.

T1.3

Question: [Applied] Which SDLC phases of Airline Crew Scheduling gain the most from AI, and which should remain human-led? Justify with risks.

AI can help during requirements analysis by grouping notes, proposing questions, and making candidate stories; it can also suggest edge cases, summarize test failures, and assist documentation. The benefits are speed and breadth, not authority.

- AI-assisted: initial note classification, candidate acceptance criteria, test ideas, code explanations, and test-failure summaries.
- Human-led: official legal interpretation, rule approval, risk acceptance, privacy decisions, fairness policy, production release, and incident response.
- Reason: inaccurate duty limits can have safety and operational impact; AI may hallucinate or omit context. Evidence includes a recorded incomplete Prompt 4 and an unsupported Boolean test expectation.

T1.4

Question: [Applied] Compare GitHub Copilot, Tabnine and ChatGPT on features, privacy, integration and suitability. Which would you recommend for Airline Crew Scheduling and why?

The tools differ by workflow, but their privacy and data handling depend on the exact product tier, account settings, organizational agreement, and deployment. Those settings were not audited for this project; do not infer a provider's privacy guarantees from the product name.

Tool	Typical workflow	Privacy / integration check	Case suitability
GitHub Copilot	Inline code completion and IDE chat.	Confirm enterprise policy, retention/training settings, repository context, and approved extension configuration.	Useful for implementation in an approved IDE; not a rule authority.
Tabnine	IDE code completion and assistant features.	Review the selected plan/deployment and data controls; verify the exact IDE integration.	Can assist code drafting where approved controls meet policy.
ChatGPT	Conversational prompting, explanation, requirements and test ideas.	Confirm account-specific privacy settings; use synthetic data only unless explicitly approved.	Required AI tool for this case; suitable for educational analysis with non-sensitive examples.

T1.5

Question: [Advanced] Describe how you would validate AI-written requirements and AI-generated test cases for Airline Crew Scheduling so that hallucinated items are caught.

Start with an authoritative rule source and give every statement a traceable ID. Split outputs into atomic requirements; classify each as supported, vague, or invented; record the source note and ask a human rule owner to resolve ambiguity. Do not silently convert an assumption into law.

- Maintain a trace matrix from each atomic rule to the authoritative source and owner; mark unresolved statements as questions rather than requirements.
- Use a reviewed test oracle, including exact boundaries (10, 10, 60), just-over/under values, simultaneous violations, and invalid inputs.
- Run generated tests against the implementation, inspect every mismatch, record accepted/rejected suggestions, and obtain human sign-off before treating a rule as operational.

Module 2 – Agile Practices and AI-Driven Planning

T2.1

Question: [Basic] Why was the Agile Manifesto created? Explain how two of its values would change the way the team builds Airline Crew Scheduling.

The Agile Manifesto was created in 2001 by software practitioners seeking an adaptive alternative to heavyweight, plan-driven delivery for changing software work. It values people and interactions, working software, customer collaboration, and responding to change, while recognizing value in the items on the right as well.

- Working software over comprehensive documentation: demonstrate one executable daily-limit slice, then expand only after its result is reviewable.
- Customer collaboration over contract negotiation: confirm duty-day, standby, and rest interpretations with the authorized rule owner instead of guessing.

T2.2

Question: [Basic] Compare Scrum, XP, Lean and Kanban in a table. Justify why Scrum is the given method for Airline Crew Scheduling.

The methods can be combined in practice, but emphasize different controls. Scrum is the assignment's fixed method; it gives a cadence for reviewing rule increments and does not imply a real multi-person team executed the ceremonies.

Method	Primary structure	Useful practice for this case	Trade-off
Scrum	Timeboxed Sprints, Product Backlog, Sprint Backlog, review and retrospective.	Review a tested constraint increment and reprioritize when a rule is clarified.	Needs active product ownership; a solo exercise simulates roles.
XP	Short development cycles with engineering discipline.	Test-first boundaries, refactoring, pair review, continuous integration.	Requires sustained engineering habits and collaboration.
Lean	Optimize flow and eliminate waste.	Limit unnecessary features; prioritize safety-relevant validation.	Less prescriptive about sprint events and roles.
Kanban	Visualize workflow and limit work in progress.	Track blocked Jenkins/Compose and pending rule sign-off visibly.	Continuous flow may provide less structured review cadence.

T2.3

Question: [Applied] Describe the sprint cycle (planning, daily scrum, review, retrospective) with timeboxes, and write a sample sprint goal for Airline Crew Scheduling.

Example for a proposed two-week Sprint (timeboxes are planning examples, not evidence that a team ran these events): planning up to four hours; a 15-minute Daily Scrum on working days; review up to two hours; retrospective up to 90 minutes. The team adapts event duration to Sprint length and need.

- Planning: select a Sprint goal, review capacity and refine stories. Daily Scrum: inspect progress toward the goal and adapt the plan. Review: inspect the increment with stakeholders. Retrospective: choose a process improvement.
- Sample goal: validate one crew-duty record against the approved daily, rest, and rolling-week rules and return an understandable result. The values used in the existing educational prototype remain provisional.

T2.4

Question: [Applied] How can AI forecast the delivery date of Airline Crew Scheduling from past sprint data? State the assumptions and the risks.

A forecast could use prior completed-Sprint velocity or throughput, a consistently defined story-point scale, historical scope changes, and uncertainty ranges. A probabilistic simulation such as Monte Carlo can sample past throughput and estimate a range of completion dates rather than a single promise.

- Assumptions: story points have a stable meaning; the same team/process has several comparable Sprints; completed work and scope changes are recorded accurately; future capacity is broadly comparable.
- Risks: small or biased history, new legal-rule work, estimates that are not independent, team changes, and false precision. Protect employee data and communicate a range, not a guaranteed date.
- No reliable historical Sprint dataset exists in this project, so no AI forecast or delivery date is claimed.

T2.5

Question: [Advanced] Describe how ChatGPT, Copilot and Jira AI features help an Agile team. List three benefits and three challenges for Airline Crew Scheduling.

ChatGPT can summarize a synthetic requirement set or propose tests; Copilot can suggest code in an IDE; Jira AI features may summarize or organize work depending on the product configuration. Each suggestion requires verification and account-level data controls.

Type	Example	Evidence / limitation
Benefit	Suggest edge and boundary cases quickly.	L1.5 records ten AI-assisted cases; one expected Boolean rejection was not supported by the rule table.
Benefit	Organize ambiguous notes into candidate requirements.	L1.4 recorded 7 supported, 3 vague, and 1 invented atomic statements from simulated notes.
Benefit	Reduce blank-page effort for planning.	L2.1–L2.3 contain mappings, a DoD, and prioritization proposals.
Challenge	AI output can be incomplete.	L3.1 Prompt 4's preserved code output is incomplete.
Challenge	AI can invent or overstate requirements.	L1.4 identifies an unsupported absolute guarantee and unresolved standby/overnight rules.
Challenge	AI estimates are not team consensus.	L2.4 still lacks three classmate estimates.

Module 3 – Development and Deployment Practices using AI Tools

T3.1

Question: [Basic] How do code-completion tools (Copilot, Tabnine) differ from chat-based tools (ChatGPT) when building duty-limit validator?

Code-completion tools primarily predict or suggest code at the cursor using nearby project context; chat-based tools accept a natural-language request and can explain, compare, or generate larger structured responses. IDE assistants may also have chat features, so the boundary is about interaction pattern rather than a strict capability split.

- Completion is most useful for local repetitive code; chat is useful for asking for explanations, alternatives, test matrices, or review of a larger snippet.
- Both can be wrong. Compare suggestions against the approved rules, review the diff, run tests, and check data-handling settings before exposing code or data.

T3.2

Question: [Applied] A prompt for duty-limit validator returned insecure code. Explain how you would refine the prompt and what review you would add.

State explicit input types/ranges, output contract, sample-rule caveat, permitted libraries, and forbidden behavior. For database use, require parameterized queries, least privilege, secret injection from protected

configuration, and no logging of personal data. Ask for code plus tests and assumptions, not claims that it was executed.

- Refine the prompt with explicit threat boundaries: validate type/range, reject bool if policy requires it, avoid string interpolation in SQL, never embed credentials, and use synthetic examples.
- Review manually; run unit and boundary tests; inspect dependencies and container configuration; run approved static/security analysis; require human approval. A prompt is not a security control.

T3.3

Question: [Basic] Explain repository, commit, branch, merge, pull request and conflict. Draw a branching strategy suitable for the team building Airline Crew Scheduling.

A repository stores project history and files; a commit is a recorded snapshot; a branch is an independent line of work; a merge integrates branches; a pull request is a reviewable proposal to merge; a conflict occurs when Git cannot automatically reconcile overlapping edits.

Branching strategy (text diagram):

```
main (protected) ← reviewed pull requests ← feature/US-01, feature/US-02, test/edge-cases.
Keep changes small; run tests before review; resolve conflicts on the branch and require a
human review before merge.
```

T3.4

Question: [Applied] Explain Jenkins jobs, pipelines, the Jenkinsfile, triggers and agents. How would a failed test stop a release of Airline Crew Scheduling?

A Jenkins job is a configured unit of automation. A pipeline is a staged workflow; a Jenkinsfile stores its pipeline definition alongside source. Triggers start it on events such as a push, pull request, or schedule. An agent is the worker that runs a stage.

- Configure checkout, dependency installation, automated tests, and package/image creation on an approved agent. A push or pull request can trigger validation; a release trigger should follow required reviews.
- The Test stage must return a non-zero status on failure. Jenkins then marks the build failed and subsequent package/publish stages should be skipped; branch protection should prevent merge/release until required checks pass.
- Jenkins build #6 actually completed Checkout, Install, Test (9 passed), dependency check, and source packaging with SUCCESS from the exact GitHub remote. A controlled failing-test/fix pair remains outstanding.

T3.5

Question: [Advanced] Discuss the benefits and challenges of AI-assisted development and deployment (CI/CD and containers) for Airline Crew Scheduling.

AI can accelerate code/test drafts and help interpret build failures. CI/CD can make repeatable checks gate a release; containers can package a consistent runtime. Together, they can improve repeatability when tests, images, and approvals are controlled.

- Benefits: reproducible build environments, earlier feedback, repeatable test gates, fewer manual packaging errors, and faster AI-assisted drafts.
- Challenges: hallucinated code, exposed secrets, unpinned or vulnerable dependencies, over-permissive images, non-deterministic suggestions, and infrastructure that is configured but not running.
- Use least privilege, pinned dependencies, secrets outside source, small non-root images, human review, and observed pipeline logs. Jenkins build #6 and the Compose/PostgreSQL demonstration have actual saved execution evidence; the controlled Jenkins failure/fix experiment is still missing.

Module 4 – AI-Based Testing and Monitoring

T4.1

Question: [Basic] Define software testing. Compare manual and automated testing and state which tests of Airline Crew Scheduling you would automate and why.

Software testing evaluates a program or system against requirements and expected behavior to discover defects and provide evidence about quality. Manual testing is performed by a person and is useful for exploration, usability, and context. Automated testing runs encoded checks repeatably.

- Automate deterministic rule boundaries, reason precedence, negative inputs, and regression cases because they are repeatable and high-value.
- Use human testing for rule interpretation, usability, workflow exploration, and operational review. The ten L4.1 direct checks are labeled manual functional invocations; automated pytest is reported separately.

T4.2

Question: [Basic] Draw the test pyramid for Airline Crew Scheduling and explain what is tested at each level and by whom.

The test pyramid places many fast unit checks at the base, fewer integration tests in the middle, and a smaller number of end-to-end/acceptance checks at the top. Test scope broadens and execution cost usually rises as the pyramid narrows.

Test pyramid (text diagram):

```
Top: acceptance / end-to-end (few; QA + product/rule owner) → integration
(API/database/browser boundaries; developers + QA) → unit (many; developers; threshold,
reason, and invalid-input logic). The unit suite and Flask API tests run locally; Selenium has
both a historical standalone fixture run and a newer live-app browser check. No database or
production acceptance test is claimed.
```

T4.3

Question: [Applied] Explain the Page Object Model and how it reduces the maintenance of Selenium tests for Airline Crew Scheduling.

Page Object Model (POM) places page structure and locator knowledge in a page-object class, exposing actions such as validate(daily, weekly, rest) to tests. Tests describe behavior rather than repeating selectors and form interactions.

- Locators and form actions are centralized in DutyValidatorPage; tests invoke page methods and assert outcomes.
- The evidence ran plain and POM scripts in Chrome against a local HTML fixture. A daily-field locator update required 3 edits in the plain script versus 1 in the POM implementation. This is a focused maintenance comparison, not a production web-app integration.

T4.4

Question: [Applied] Explain AI-assisted test case generation and test prioritisation. How would you prioritise tests for Airline Crew Scheduling before a release?

AI-assisted test generation proposes cases from requirements, code, or failure history. Generated cases must be checked against the approved rule table because a fluent expected result can still encode an unsupported rule.

- Generate candidate cases from traceable rules, then remove expectations not supported by the case specification and add missing boundaries or negative positions.
- Prioritize safety-impacting rule paths, recent changes, boundary and combined violations, then broader UI/regression checks. Re-run the full suite before release.
- L4.4 scored 9 of 10 generated designs as supported by the rule table (90%). The Boolean expectation was excluded because the assignment rule table does not define a Boolean input policy.

T4.5

Question: [Advanced] Explain defect prediction and AI-based bug detection. Which data would you collect for Airline Crew Scheduling, and how would the team act on the predictions?

Defect prediction uses historical engineering signals to estimate where defects may occur; AI bug detection can identify suspicious patterns in code or tests. Neither prediction proves a defect exists, and a low score does not prove safety.

- Potential signals: defect history, changed lines/code churn, complexity, test gaps, failed builds, review findings, dependency alerts, and anonymized operational incidents.
- Use predictions to prioritize human review and tests, monitor precision/recall, check for bias, and avoid using scores to blame individual developers. A human confirms defects and approves remediation.
- This project has no validated defect dataset or measured prediction model; defect prediction is a proposed practice only.

Module 5 – AI-Powered Quality Assurance and Ethical Development

T5.1

Question: [Basic] For Airline Crew Scheduling, which two quality attributes matter most and why? How would you measure them?

Two top attributes are correctness/safety and reliability. Incorrect validation may accept an unlawful schedule or reject a legal one; unreliable behavior can disrupt planning and operations. Both require approved requirements and measurable tests.

Attribute	How to measure in this case
Correctness / safety	Trace each decision to an approved rule; test every boundary and combined violation; count false accepts/rejections in a validated test set. No test alone proves legal compliance.
Reliability	Measure failure rate, availability, recovery time, and reproducibility under defined operating conditions. No production service or reliability SLO is present in the student prototype.

T5.2

Question: [Applied] What is SonarQube? Explain bugs, vulnerabilities, code smells, duplication, coverage and quality gate in the context of Airline Crew Scheduling.

SonarQube is a code-quality and security analysis platform that inspects supported source for issues and reports quality measures. Exact rules and features depend on version, language analyzer, and configured quality profile.

SonarQube concept	Case-specific interpretation
Bug	A likely defect that could cause incorrect validator behavior.
Vulnerability	A security weakness that could expose or alter crew/scheduling data.
Code smell	Maintainability concern that raises change risk.
Duplication	Repeated rule logic that can drift when limits change.
Coverage	How much code tests execute; not a measure of rule correctness.
Quality gate	Configured pass/fail criteria before merge/release; threshold must be set by the team.

No SonarQube scan or gate was run for this project. L5.2 reports Pylint findings only; they must not be relabeled as SonarQube.

T5.3

Question: [Applied] Compare AI code review with human review and static analysis for Airline Crew Scheduling. How would you combine the three?

AI review can rapidly suggest likely defects or readability improvements, but can miss context or invent concerns. Human review evaluates requirements, architecture, safety, and trade-offs, and remains accountable. Static analysis deterministically applies configured rules to code.

- AI review is a fast second opinion, not a verified diff assessment; human review checks domain meaning and owns decisions; static analysis applies configured rules consistently.
- Combine them by running analysis and tests, having AI suggest review questions, then asking a human reviewer to validate each finding against source requirements. Record accepted/rejected suggestions and block release on unresolved high-risk findings.
- L5.3 locally reviewed the actual merged PR #7 diff and verified its suggestions; GitHub recorded zero posted review comments. Local notes are not a posted PR review or human approval.

T5.4

Question: [Advanced] Who is accountable when AI-generated code in Airline Crew Scheduling causes harm? Discuss the developer, the company and the tool vendor.

The developer is accountable for professional review, testing, disclosure of known limitations, and refusing unverified safety-critical code. The organization is responsible for governance, approved rule sources, access controls, release decisions, monitoring, and incident response.

- Developer: review generated code, understand it, test it, document limitations, and stop unsafe changes.
- Company: define approved rules, privacy controls, review gates, training, monitoring, and incident response; it remains responsible for deployment decisions.
- Vendor: meet contractual and legal obligations for the service and represent its capabilities accurately; vendor terms do not transfer the deployer's operational responsibility.

T5.5

Question: [Advanced] Discuss benefits and challenges of AI in SQA for Airline Crew Scheduling. Why must AI assistance be acknowledged?

AI can suggest test cases, summarize code changes, surface suspicious patterns, and reduce repetitive review effort. It may also produce false positives, false negatives, biased evaluations, or confident but unsupported requirements. Its performance depends on context and validation.

- Benefits include broader test ideas, faster repetitive analysis, and easier test documentation. Challenges include false alarms, missed defects, biased data, leakage, unclear provenance, and over-reliance.
- Disclose the tool and purpose, preserve the human verification step, and identify what was changed or rejected. Acknowledgment supports academic integrity, reproducibility, and accountable review.
- Appendix A records the known AI use and verification limits for this submission.

Part 3 – Laboratory Exercises

The following 25 status labels follow the latest authoritative matrix in CASE57_EVIDENCE_AUDIT.md. The specific evidence records in CASE57_LAB_EVIDENCE provide the supporting details; stale historical judgments are not used.

Current status	Count
COMPLETE	17
PARTIAL	5
PENDING HUMAN	3
BLOCKED	0

Module 1 Labs – SDLC, Process Models and AI Tools

L1.1

Question: [Basic] Draw the SDLC phases for Airline Crew Scheduling and write one output per phase. Submit: diagram and table.

CURRENT STATUS: COMPLETE — The seven phases and outputs are documented: planning (scope/risk), requirements (approved constraints), design (validator flow), implementation (duty_validator.py), testing (two test files/results), deployment (package guidance), and maintenance (defect/rule updates). Testing feedback loops to requirements when a rule is unclear. The 10/10/60 values remain sample assumptions.

SDLC phase	Deliverable
Planning	Scope, stakeholder/risk list, milestone
Requirements	Approved rules and I/O contract
Design	Validator flow and rule precedence
Implementation	duty_validator.py
Testing	Unit and AI-labeled cases/results
Deployment	Packaged prototype instructions
Maintenance	Defect log and approved rule updates

L1.2

Question: [Applied] Plan the first milestone of Airline Crew Scheduling under Waterfall and under Incremental and compare effort, risk and feedback. Submit: comparison table.

CURRENT STATUS: COMPLETE — The Waterfall estimate is 14 hours (requirements 3, design 2, implementation 4, testing 3, packaging/review 2). The three-increment plan is 4 + 5 + 5 = 14 estimated hours. Incremental feedback occurs after each slice and can expose rule questions earlier. These are planning estimates, not measured durations.

L1.3

Question: [Applied] Run the same prompt for duty-limit validator in two AI tools and compare correctness, explanation and privacy aspects. Submit: outputs and a table.

CURRENT STATUS: PARTIAL — Preserved ChatGPT and Copilot SDK outputs plus the project validator were replayed on the same 14 cases; all matched the expected semantic classifications 14/14. Copilot is attributed only to Copilot. Account-specific privacy settings and plan remain unverified, so the privacy comparison is incomplete. Synthetic rules only; no crew personal data.

Dimension	ChatGPT	Copilot SDK assistant
Output completeness	Docstring/types included in preserved result.	Compact response omitted requested annotations/docstring.
Ten semantic cases	10/10 matched the paired output.	10/10 matched the paired output.
Privacy	Account settings not checked.	Account/workspace settings not checked.

L1.4

Question: [Applied] Write 8 lines of messy client notes for Airline Crew Scheduling. Ask AI to extract requirements. Classify each result as correct, vague or invented. Submit: notes, AI output, corrected list with counts.

CURRENT STATUS: COMPLETE — Eight simulated notes were used, not real client statements. The recorded extraction contains 11 atomic statements: 7 supported, 3 vague, and 1 invented absolute guarantee. Standby treatment, duty-day boundaries, and “very quickly” need clarification. The 400-crew scenario is not a capacity guarantee.

Classification	Count	Interpretation
Supported	7	Supported by simulated notes, not independently verified law.
Vague	3	Duty-day boundary, standby treatment, measurable speed.
Invented	1	Unsupported absolute guarantee that a roster is never wrong.

L1.5

Question: [Advanced] Generate 10 test cases for duty-limit validator with AI, run them against your code and report which failed and why. Submit: results table.

CURRENT STATUS: COMPLETE — Actual ChatGPT tests were run against project code: 9/10 matched. TC10 expected a Boolean to raise ValueError, but the project accepts True; the case's rule table does not itself specify a Boolean policy. Supplementary string and NaN/infinity probes exposed additional input-hardening gaps. The mismatch is disclosed; no code change is claimed.

Case group	Result
First ten AI tests	9 matched; Boolean case expected ValueError but project accepted True.
Supplementary string	TypeError rather than expected ValueError.
Supplementary NaN	Accepted unexpectedly.
Supplementary positive infinity	Weekly rejection rather than expected ValueError.
Negative infinity in rest	ValueError as expected.

Module 2 Labs – Agile, Scrum and AI-Driven Planning

L2.1

Question: [Basic] Write 6 statements describing practices in Airline Crew Scheduling and classify each under an Agile Manifesto value or principle. Submit: table.

CURRENT STATUS: COMPLETE — Six proposed practices map to working software, customer collaboration, responding to change, technical excellence, welcoming changes, and individuals/interactions. They are planning artifacts; no functioning Scrum team or ceremony execution is claimed.

L2.2

Question: [Basic] Write a sprint goal and a Definition of Done for the first iteration of Airline Crew Scheduling. Submit: both, with 3 acceptance checks.

CURRENT STATUS: COMPLETE — Goal: validate three configurable sample constraints, reject negatives, and explain decisions. DoD covers documented assumptions, inclusive boundaries, negative-input ValueError, passing automated cases, and no legal-compliance overclaim. Three direct acceptance checks (daily boundary, rest rejection, weekly rejection) passed; this was not team sign-off.

L2.3

Question: [Applied] Prioritise the backlog of Airline Crew Scheduling by MoSCoW and by value versus effort. Submit: two tables and an explanation of the differences.

CURRENT STATUS: COMPLETE — Eight stories were ranked. US-01–US-04 are Must; input validation is Should; decision log and roster import are Could; fairness reporting is deferred for the first prototype. Value/effort ratings are planning judgments, not empirical or team votes. A high future value does not make roster generation part of the current implementation.

ID	Story	MoSCoW	Value/Effort
US-01	Daily duty check	Must	5 / 1
US-02	Minimum rest check	Must	5 / 1
US-03	Rolling-week check	Must	5 / 2
US-04	Explain rejection	Must	4 / 1
US-05	Validate input types/ranges	Should	4 / 2
US-06	Decision audit trail	Could	3 / 3
US-07	Import crew roster	Could	3 / 4
US-08	Fairness/coverage report	Won't for first prototype	4 / 5

L2.4

Question: [Applied] Estimate the 10 stories with planning poker (or with a team of 3) and with AI. Calculate the difference for each story. Submit: table and reasons for the biggest gap.

CURRENT STATUS: PENDING HUMAN — Ten stories and actual ChatGPT-only Fibonacci estimates (total 56 points) are recorded. A separate simulation by one Copilot SDK assistant produced three role-based perspectives: simulated medians total 64, with a largest simulated gap of 5 points on US-08. These are not people or independent runs. Human 1/2/3 columns, actual team differences, and real discussion remain pending.

Story	ChatGPT AI points	Simulated role estimates	Simulated median / gap
US-01 daily check	2	2 / 3 / 2	2 / 0
US-02 minimum rest	2	2 / 2 / 3	2 / 0
US-03 rolling week	5	5 / 3 / 5	5 / 0
US-04 rejection reasons	3	3 / 3 / 2	3 / 0
US-05 input types/ranges	3	5 / 3 / 5	5 / 2
US-06 decision audit	5	5 / 8 / 8	8 / 3
US-07 roster import	5	8 / 5 / 8	8 / 3
US-08 overnight interpretation	8	13 / 13 / 8	13 / 5
US-09 Compose/database	5	5 / 5 / 8	5 / 0
US-10 fairness evaluation	13	13 / 8 / 13	13 / 0

The three role columns are simulated by one Copilot SDK assistant, not three people or independent runs. Simulated medians total 64; the largest simulated gap is 5 points on US-08. The official human comparison still cannot be computed: three real classmate estimates, median differences, and discussion remain PENDING HUMAN.

L2.5

Question: [Advanced] Write 3 benefits and 3 challenges of using AI in the Agile work of Airline Crew Scheduling, each supported by evidence from your lab results. Submit: table.

CURRENT STATUS: COMPLETE — Three evidence-linked benefits: test ideas (L1.5), note classification (L1.4), planning support (L2.1–L2.3). Three challenges: incomplete code (L3.1 Prompt 4), invented/vague requirements (L1.4), and AI estimates not replacing team consensus (L2.4). Each is paired with an appropriate control.

Module 3 Labs – Prompting, Git, Jenkins and Docker

L3.1

Question: [Basic] Write four versions of a prompt for duty-limit validator, each better than the last. Score each output from 1 to 5. Submit: prompts, outputs and score table.

CURRENT STATUS: PARTIAL — Four actual ChatGPT prompt/output records and a five-criterion score table are preserved. Totals: Prompt 1 15/25, Prompt 2 22/25, Prompt 3 25/25, Prompt 4 8/25; the original ChatGPT Prompt 4 code is incomplete. A separately attributed Copilot SDK retry passed its 12 displayed examples. This does not complete the missing ChatGPT response or change its attribution. Figure 1 is a genuine saved ChatGPT screenshot.

Prompt	Score	Evidence quality
1	15/25	Rules described; return contract/input robustness absent.
2	22/25	Tuple and boundaries addressed; only negative validation.
3	25/25	Complete preserved output; not independently executed.
4	8/25	Most specific prompt, but retained code is incomplete.

Separate Copilot SDK retry	Actual result	Attribution boundary
12 displayed examples	12 passed	Copilot retry only; does not complete or replace the incomplete original ChatGPT Prompt 4.

L3.2

Question: [Applied] Ask AI to refactor one function of duty-limit validator, then prove the behaviour is unchanged with tests. Submit: before, after and test output.

CURRENT STATUS: COMPLETE — The negative-input comparisons were consolidated into a loop over the three inputs. The documented suite passed 20 tests before and after; nine direct before/after cases matched, including exact boundaries, precedence and each negative-input position. The validation order and return behavior were preserved. No new screenshot was captured.

Before: three repeated negative checks in one condition. After: iterate the tuple of three inputs and raise the same ValueError for any negative value. The rule checks, precedence, reasons, boundaries, and return tuples are unchanged.

```
# Before
if hours_today < 0 or hours_last_7_days < 0 or rest_hours < 0:
raise ValueError("Duty hours and rest hours cannot be negative.")
# After
for hours in (hours_today, hours_last_7_days, rest_hours):
if hours < 0:
raise ValueError("Duty hours and rest hours cannot be negative.")
```

Run	Recorded result
Before refactor	20 passed in 0.08s
After refactor	20 passed in 0.05s
Direct behavior comparisons	9/9 matched

L3.3

Question: [Basic] Create and resolve a merge conflict in duty-limit validator. Submit: conflict markers, your resolution and what AI suggested.

CURRENT STATUS: COMPLETE — A temporary isolated repository created conflicting edits to the same success-message line. Actual conflict, AI suggestion, resolution, two-parent merge commit 80da4aa…, clean status, and smoke checks are documented. The original project branch/history was not changed; no conflict was created in this repository.

Actual conflict alternatives were:

```
<<<<<<< HEAD
return True, "Duty schedule accepted under sample rules."
=======
return True, "Duty schedule is within limits."
>>>>>>> feature/l3.3-success-message
```

Recorded AI suggestion: return True, "Duty schedule accepted under sample limits." The resolution was performed and committed in the temporary repo as 80da4aa…; project history was untouched.

L3.4

Question: [Applied] Write a Jenkinsfile with checkout, build, test and package stages for duty-limit validator. Show a successful build and a failed build with its console log and fix. Submit: file and screenshots.

CURRENT STATUS: PARTIAL — Jenkins build #6 actually checked out the exact two-dot GitHub remote, installed dependencies, ran 9 tests successfully, completed pip check and source packaging, and finished SUCCESS; genuine logs/screenshots are preserved. The deliberate failing-test followed by corrected-success build pair has not been run, and Jenkins is currently unavailable. A separate-branch procedure is documented.

Build #6 stage	Actual result
Checkout	SUCCESS from https://github.com/manojshet2005/Airline-Crew-Scheduling..git; feature/weekly-limit.
Install and Build	SUCCESS; dependencies installed; compileall completed.
Test	9 passed in 0.02s.
Quality Check and Package	pip check clean; duty-validator.zip archived.
Final result	SUCCESS; actual log and screenshots listed in the evidence index.

Remaining official requirement: a deliberately failing test/build followed by a corrected successful build with console evidence. Jenkins is currently unavailable, so that controlled pair was not run. Build #3 was a pipeline parser failure and is not substituted for the required test failure.

L3.5

Question: [Advanced] Run duty-limit validator with a database using Docker Compose and show both containers communicating. Submit: compose file and logs.

CURRENT STATUS: COMPLETE — A genuine Compose run built the validator image, connected the one-shot validator to healthy PostgreSQL, wrote validation rows, and verified persisted records. Execution logs and a real

terminal screenshot are preserved. The validator exits after its run; this is not a persistent production service. Docker Engine is not currently available, but historical execution is evidenced.

Compose evidence	Actual result
Image/build and database	Image built; PostgreSQL became healthy.
Validator execution	Connected successfully, returned accepted result, and stored one row.
Independent rerun	Connected successfully; total persisted rows verified as 2.
Container behavior	Validator exited 0 after its one-shot task; PostgreSQL remained healthy.
Evidence	compose_execution.log and compose_live_verification.png are genuine run evidence.

This proves a one-shot validator/database demonstration, not a persistent service or production deployment. Docker Engine is not currently available, but the historical execution is complete and preserved.

Module 4 Labs – Testing, Selenium and Monitoring

L4.1

Question: [Basic] Write 10 manual test cases for duty-limit validator, execute them, and report 2 defects (real or seeded) with steps, expected result, actual result and severity. Submit: tables.

CURRENT STATUS: COMPLETE — Ten direct invocations against the project function passed, including boundaries, rule precedence, and all three negative positions. Two intentionally faulty functions existed only in a temporary inline test: daily >=10 wrongly rejected exactly 10; weekly >=60 wrongly rejected exactly 60. Both seeded defects were reproduced; each was rated medium. Project source was not modified for the seeded cases.

TC	Input	Expected / actual
01	(8,40,12)	Accept / Accept
02	(10,60,10)	Accept / Accept
03	(10.01,40,12)	Daily reject / Daily reject
04	(8,40,9.99)	Rest reject / Rest reject
05	(8,60.01,12)	Weekly reject / Weekly reject
06	(11,40,9)	Daily reason first / Daily reason
07	(8,61,9)	Rest reason first / Rest reason
08–10	Each input position negative	ValueError / ValueError

Seeded defect	Expected	Observed	Severity
Daily >= 10 instead of > 10	Accept exactly 10	Rejected	Medium
Weekly >= 60 instead of > 60	Accept exactly 60	Rejected	Medium

L4.2

Question: [Basic] Write 5 acceptance scenarios for Airline Crew Scheduling in Given-When-Then form and execute them with a classmate as the client. Submit: scenarios and sign-off.

CURRENT STATUS: PENDING HUMAN — Five technical scenarios were executed and a separate fictional stakeholder role simulation documents questions and conditional classroom-only criteria. One assistant generated the role; this is not a classmate/client or sign-off. Real client/classmate decisions and signature/date remain pending.

ID	Scenario	Executed technical result	Actual client decision
A1	10 daily, 40 weekly, 10 rest → accept	Passed	PENDING HUMAN
A2	10.01 daily → daily rejection	Passed	PENDING HUMAN
A3	9.99 rest → rest rejection	Passed	PENDING HUMAN
A4	60.01 weekly → weekly rejection	Passed	PENDING HUMAN
A5	Negative input → validation error	Passed; API returned HTTP 400	PENDING HUMAN

One AI assistant also generated a fictional operations-stakeholder perspective and conditional classroom-only criteria. This is a simulation, not a real client decision. Client/classmate, acceptance decision, signature, and date: [PENDING HUMAN].

L4.3

Question: [Applied] Convert your Selenium script to the Page Object Model and compare the maintenance effort. Submit: both versions and a note.

CURRENT STATUS: COMPLETE — The earlier plain and POM scripts ran in Chrome via Selenium Manager against the local HTML fixture; boundary and daily-violation assertions passed in both. The live Flask dashboard was added later and independently exercised in Chrome for valid, daily/rest/weekly violations, reset, missing-input feedback, and a 390-pixel viewport. The original fixture screenshots remain Figures 2–3; the newer live application screenshots are Figures 4–5. A daily-field locator change in the fixture required three edits in the plain script versus one in the POM.

Script	Actual run
Plain Selenium	Chrome: boundary PASS; daily violation PASS; exit 0.
Page Object Model	Chrome: boundary PASS; daily violation PASS; exit 0.
Locator maintenance	3 edits in plain script vs 1 centralized POM constant.

The screenshot captures show the accepted (10, 60, 10) fixture state; the 10.5 daily violation was asserted in the same run.

L4.4

Question: [Applied] Ask AI for tests for Airline Crew Scheduling, keep the valid ones, remove the wrong ones and add missing ones. Submit: accuracy percentage and the final set.

CURRENT STATUS: COMPLETE — Nine of ten initial AI test designs were supported by the rule table: 90%. The Boolean-to-ValueError expectation was removed because the assignment rules do not specify Boolean handling; negative daily, weekly, and rest positions were retained/covered in the final ten. The rating is test-design validity, not a legal correctness or full suite pass rate.

Metric	Value
AI case designs supported by case rule table	9 / 10 = 90%
Unsupported case removed	Boolean must raise ValueError (type policy absent from assignment rules)
Final negative input coverage	Daily, weekly, and rest positions

L4.5

Question: [Advanced] Design a monitoring dashboard for Airline Crew Scheduling with 6 metrics, thresholds and alert actions. Submit: sketch and table.

CURRENT STATUS: COMPLETE — A six-card dashboard sketch and threshold/action table exist. All example values are mock, not live metrics. The application does not currently emit or aggregate these measurements.

Metric	Mock value	Threshold	Alert action
Validator error rate	1.2%	>2%	Notify developers; investigate validation errors.
Response time (p95)	320 ms	>500 ms	Notify operations; investigate slow requests.
Daily rule violations	12	>20	Notify scheduling supervisor; review rejections.
Failed Jenkins builds	0	2 consecutive failures	Notify developers; inspect pipeline logs.
Test pass rate	98.7%	<95%	Notify QA; investigate failing tests.
Crew data age	8 min	>15 min	Notify data team; refresh crew information.

All values are mock design values; no live monitor, Jenkins, or crew-data telemetry is connected.

Module 5 Labs – SQA, SonarQube and Ethical AI

L5.1

Question: [Basic] Review a classmate's code or requirement list for Airline Crew Scheduling with a checklist and report the defects found. Submit: checklist and findings.

CURRENT STATUS: PENDING HUMAN — Three role-based AI review perspectives and nine focused source checks are documented as a simulation; they are not independent reviewers. No classmate artifact or participant was supplied and no human findings are recorded. Reviewer, artifact/commit, genuine findings, and confirmation/date remain pending.

- Trace each requirement to an approved source.

- Check daily/rest/weekly boundaries and invalid inputs.
- Verify reason priority, test coverage, privacy, and sample-rule caveat.
- Preserve the public function contract.

AI role perspective	Evidence-grounded observation	Disposition
Domain correctness	10/10/60 and daily → rest → weekly order are sample assumptions, not verified law.	Blocking scope limitation; obtain authoritative rule approval.
Maintainability	Thresholds are literals in the shared validator.	Low improvement; no extra configuration added.
Security/operations	Flask is an academic prototype; Compose demo fallback secret is source-visible.	Deployment risk, not an exploit finding; no production claim.

One Copilot SDK assistant generated all three role perspectives and focused checks. They are AI simulation, not three peer reviewers. No human peer findings are recorded.

Peer name, reviewed artifact/commit, findings, and reviewer confirmation/date: [PENDING HUMAN].

L5.2

Question: [Applied] Explain three SonarQube findings in detail (rule, risk, fix). Submit: report.

CURRENT STATUS: PARTIAL — No SonarQube server/scanner was available. Three actual Pylint findings are distinguished as alternatives: C0114 missing module docstring, C0116 missing test-function docstrings, and C0103 invalid test names. Pylint exited 16 with 1.25/10. Risks and suggested fixes are documented; these are not SonarQube findings.

Pylint rule (not SonarQube)	Finding	Risk / proposed fix
C0114	Missing module docstring	Scope less clear; add a concise docstring.
C0116	Missing test-function docstrings	Intent less discoverable; add behavior names/docstrings or set test-file policy.
C0103	test_TC_01-style names	Style inconsistency; use descriptive snake_case names.

Recorded test-file Pylint result: exit 16, score 1.25/10. Separate current checks gave duty_validator.py 10.00/10, scoped Flake8 no findings, and scoped Bandit no findings. These tool/scope results are distinct; none is SonarQube. The scanner, server, token, and Docker engine were unavailable, so no SonarQube scan was claimed.

L5.3

Question: [Applied] Ask AI to review a pull request of Airline Crew Scheduling and test every suggestion before accepting it. Submit: suggestion log with results.

CURRENT STATUS: COMPLETE — The actual merged PR #7 diff was reviewed locally with AI assistance; suggestions were tested against code and the 36-test suite. GitHub has zero posted review comments. Local notes are not presented as posted PR feedback or human approval.

Suggestion	Verification	Decision
Cover negative input positions	Inspected code/tests; full suite passed.	Already satisfied; no change.
Use descriptive test names	Pylint C0103 confirmed.	Valid; deferred to preserve existing test names.
Check expectations against approved rules	Reviewed caveats and L1.5 cases.	Accepted as process guidance.

The reviewed artifact was the actual merged PR #7 diff, reviewed locally. GitHub has zero posted review comments. This local suggestion log is not a posted PR review or human approval.

L5.4

Question: [Advanced] Write one function yourself and one with AI for duty-limit validator; run 10 tests and SonarQube on both. Submit: comparison table.

CURRENT STATUS: PARTIAL — The project implementation passed 10/10 shared cases; the AI reference passed 8/10 and failed Boolean/NaN validation. Three seven-round runtime microbenchmark runs are preserved, with methodology, samples/ranges, and limitations; these are not authoring-time or production-performance measurements. SonarQube scans of both implementations remain missing.

Comparison dimension	Observed result
Required shared rule cases	Project implementation: 10/10; Copilot AI reference: 8/10 (Boolean and NaN failures).
Three runtime runs	Seven rounds each; project medians 1085.5, 935.9, 927.2 ns/call; AI medians 871.9, 844.0, 830.6 ns/call.

Comparison dimension	Observed result
Measurement limitation	Short in-process microbenchmarks only; not authoring time, endpoint latency, or 400-crew performance.
SonarQube	No scanner/server/token/Engine; scans of both versions remain missing.
Authorship provenance	Original project authoring time was not independently verified.

L5.5

Question: [Advanced] Write the AI disclosure statement and a one-page AI usage policy for the Airline Crew Scheduling team, and check the licence of one library or AI snippet you used. Submit: documents.

CURRENT STATUS: COMPLETE — Disclosure and a one-page policy are included in Appendix A. Flask 3.1.3 package metadata and its BSD-3-Clause license text were inspected; redistribution conditions/notice retention are noted. pip-licenses was not run and no output is claimed.

AI disclosure and the one-page project usage policy are in Appendix A. The inspected library was Flask 3.1.3, BSD-3-Clause; license conditions and notice requirements are recorded in the evidence file. This is not legal advice.

References

Reference	Use
AI in Software Engineering Assignment Book (75 Cases), Case 57, printed pp. 228–231.	Official tasks, theory wording, labs, and appendices.
AI in Software Engineering Individual Assignment Workbook.	Supplementary submission, AI-use, and evidence guidance.
AI in Software Engineering Lab Manual.	Supplementary laboratory method and evidence guidance.
CASE57_INSTRUCTIONS.md	Project facts, fixed choices, test summary, and evidence rules.
CASE57_EVIDENCE_AUDIT.md	Authoritative current status matrix and audit scope.
CASE57_PPT_CONTENT_BLUEPRINT.md	Visual guidance and evidence guardrails; deck adapted to requested 13 slides.
Case45_EGovernance_Grievance_Portal_Updated_Tanmay.pptx	User-provided visual reference only; no presentation content was reused.
duty_validator.py; test_duty_validator.py; test_ai_generated.py	Implementation and current test sources.
CASE57_LAB_EVIDENCE/L1.1_Evidence.md–L5.5_Evidence.md	Per-lab prompts, outputs, execution records, limitations, and status.
GitHub repository	Student-confirmed repository page and verified two-dot Git transport URL; origin was read successfully on feature/weekly-limit.
Python documentation	https://docs.python.org/3/ — language and standard-library reference.
Flask documentation	https://flask.palletsprojects.com/ — application and test-client reference.
Selenium documentation	https://www.selenium.dev/documentation/ — browser automation reference.
Git documentation	https://git-scm.com/docs — repository and version-control reference.
Docker Compose documentation	https://docs.docker.com/compose/ — reference for the executed one-shot Compose/PostgreSQL demonstration; not a production deployment.
Jenkins documentation	https://www.jenkins.io/doc/ — prepared pipeline workflow reference; no Jenkins run is claimed.

Appendix A – AI Usage Log

The following log distinguishes recorded prior AI activity from this final preparation. Prompts are summarized unless a specific actual prompt is reproduced in the lab evidence. No real crew personal data or secrets were intentionally included.

Step	Tool	Purpose / prompt summary	Verification	Decision
1	ChatGPT (prior shared session)	Duty validator prompts, simulated-note extraction, test cases, and AI-only estimates.	Compared outputs with assignment rules, project code, and recorded direct runs; L1.3 privacy settings remain unverified.	Accepted supported material; recorded mismatches, vague items, and incomplete output.
2	Copilot SDK assistant in VS Code	Paired L1.3 response, planning artifacts, local review notes, and clearly labeled AI role simulations for L2.4, L4.2, and L5.1.	Checked simulation claims against evidence and code; ran focused scenario/review checks and the project test suite where recorded.	Simulations are not independent models, classmates, client acceptance, posted PR comments, or human peer findings.
3	Copilot SDK assistant in VS Code	Prepared this report and 13-slide presentation from the official PDF, project sources, and audit.	Read exact Case 57 assignment pages; checked 25 lab status rows; verified original screenshots and actual Jenkins/Compose logs.	Submitted with clear status labels, source caveats, and placeholders.

One-page AI usage policy for the Case 57 team

Policy area	Team rule
Permitted use	Use AI to draft non-sensitive requirements, tests, code explanations, documentation, and review questions; treat output as a suggestion.
Data protection	Never submit crew identifiers, confidential rosters, credentials, production logs, or other non-public airline data. Use synthetic examples and verify tool privacy settings.
Verification	Trace every rule to an approved source; inspect generated code; run boundary/invalid-input tests; independently verify every review recommendation.
Human accountability	A named human approves rules, releases, fairness criteria, and operational decisions. AI does not make unreviewed crew assignments.
Disclosure and records	Record tool/provider, purpose, prompt/output summary, verification, decisions, and reviewer; disclose AI assistance in academic work.
Licensing and security	Check third-party and AI-snippet licenses; preserve notices; use least privilege and protected secret management; never commit secrets.

Appendix B – AI Disclosure Statement and Supporting Evidence

Official AI Disclosure Statement

I used ChatGPT and the Copilot SDK assistant in VS Code to help draft and review project code, tests, planning material, this report, and the presentation. One Copilot SDK assistant also generated explicitly simulated estimator, airline-stakeholder, and reviewer-role perspectives for L2.4, L4.2, and L5.1; these are not human participation or independent model outputs. I verified the submitted material against the official Case 57 assignment, repository files, recorded lab evidence, and the current project test run (36 passed). No real crew personal data, passwords, or tokens were intentionally shared in the AI interactions documented for this project.

Student signature: _____ Date: _____

Figures below are genuine images from the project evidence folder. The L3.1 image shows a saved ChatGPT response view and is not proof of test execution. Figures 2–3 show the historical Selenium fixture; Figures 4–5 show the later live Flask application; Figures 6–7 show actual Jenkins build #6 console results; Figure 8 shows the real Compose/PostgreSQL verification. The original 22 screenshots were reviewed individually; the historical Flask debugger screenshot is excluded because it contains a visible local debugger credential. No missing screenshot or infrastructure output has been fabricated.

L3.1 Prompt 3 output is referenced in the L3.1 result; the retained response was not independently executed.

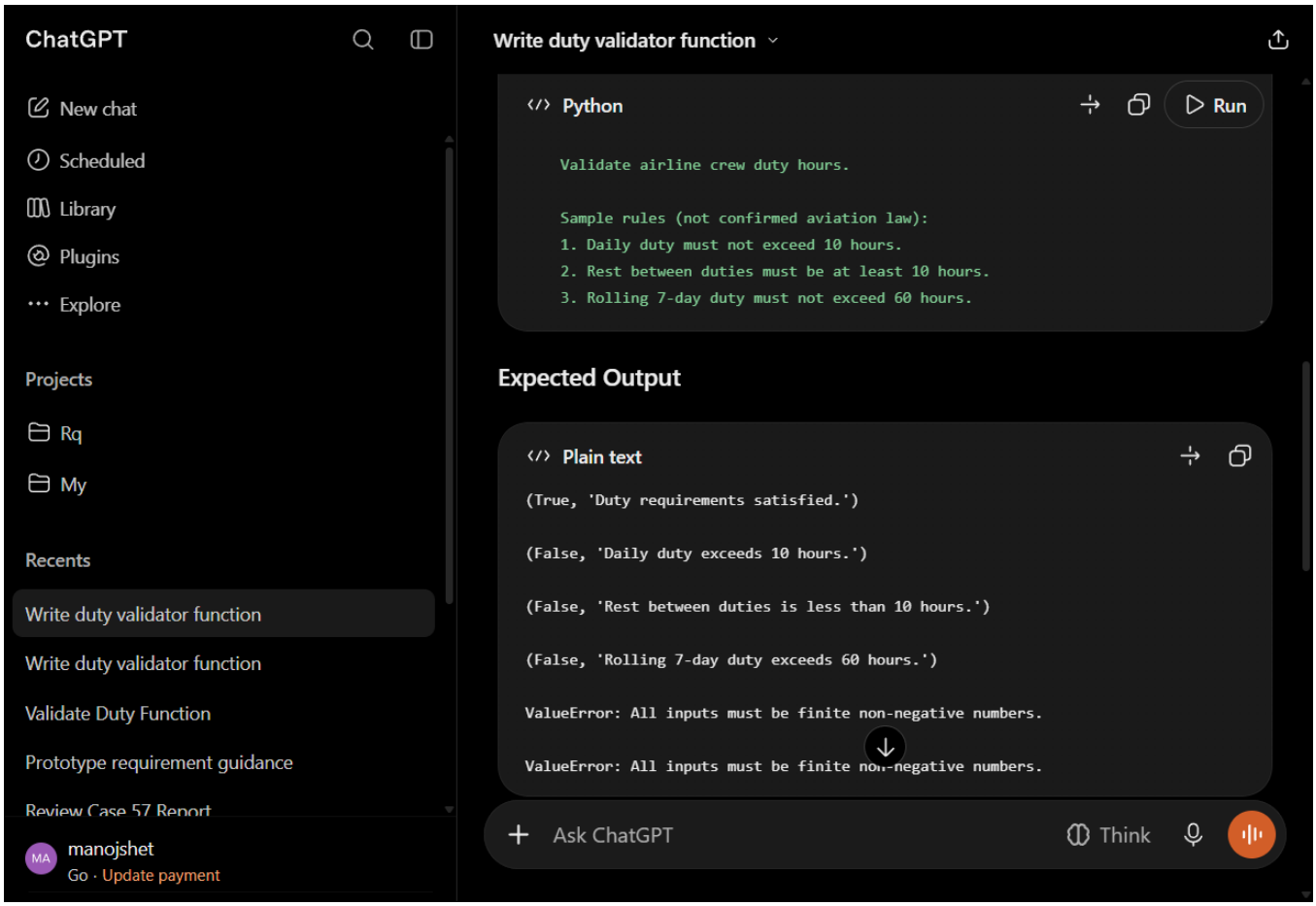


Figure 1. Saved ChatGPT Prompt 3 response screenshot (L3.1); source output is evidence of a response, not an execution result.

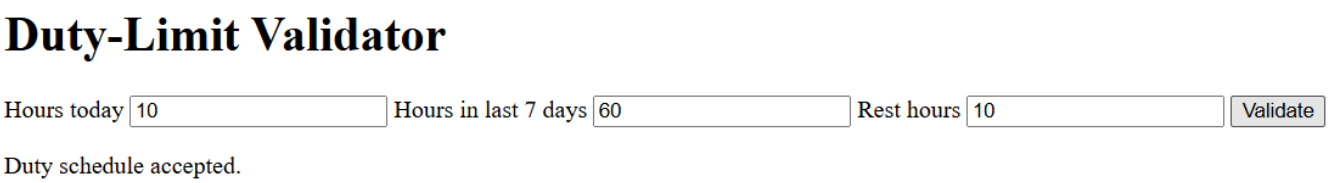


Figure 2. Cropped genuine Chrome fixture screenshot from the plain Selenium run (L4.3), showing the inclusive-boundary result.

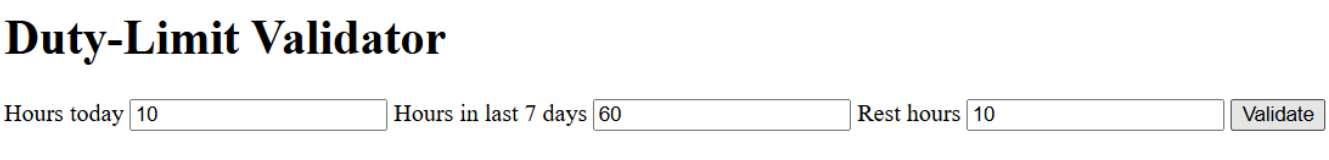


Figure 3. Cropped genuine Chrome fixture screenshot from the Page Object Model run (L4.3), showing the inclusive-boundary result.

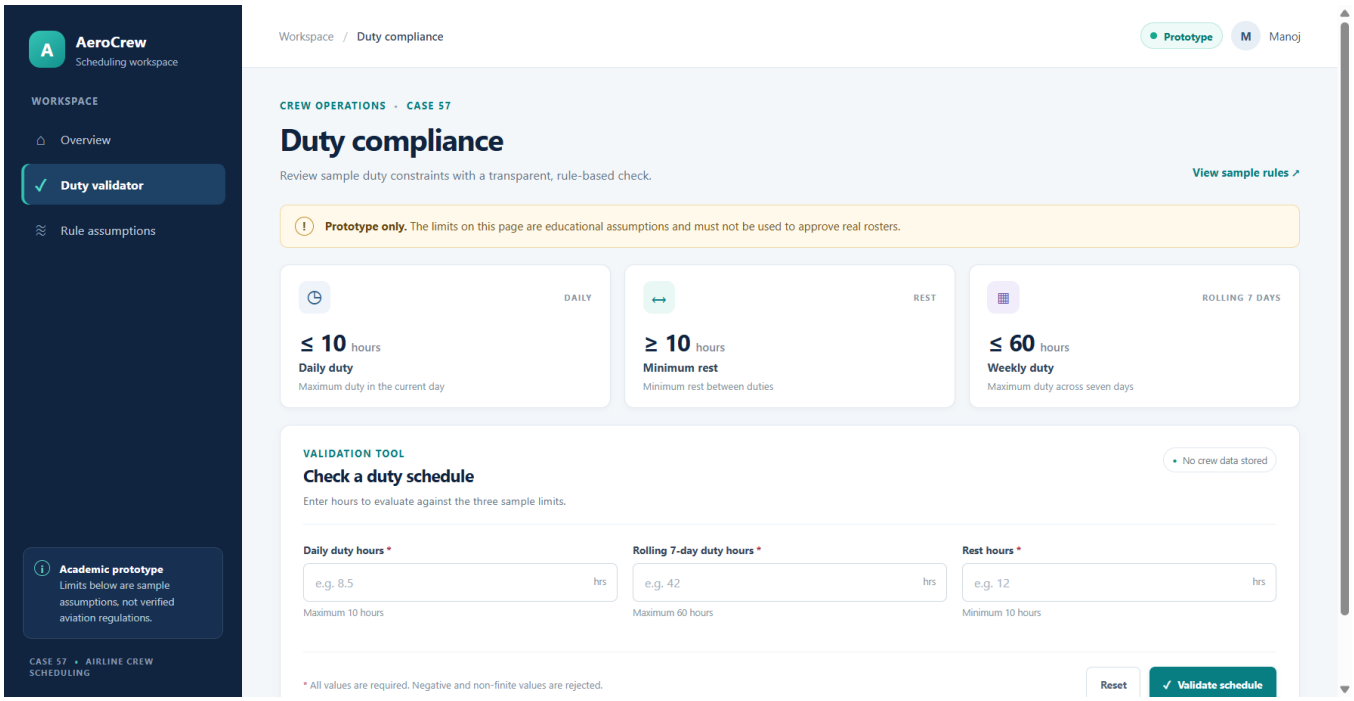


Figure 4. Live Flask duty dashboard captured in Chrome at desktop width; sample-rule caveat and validator inputs are visible.

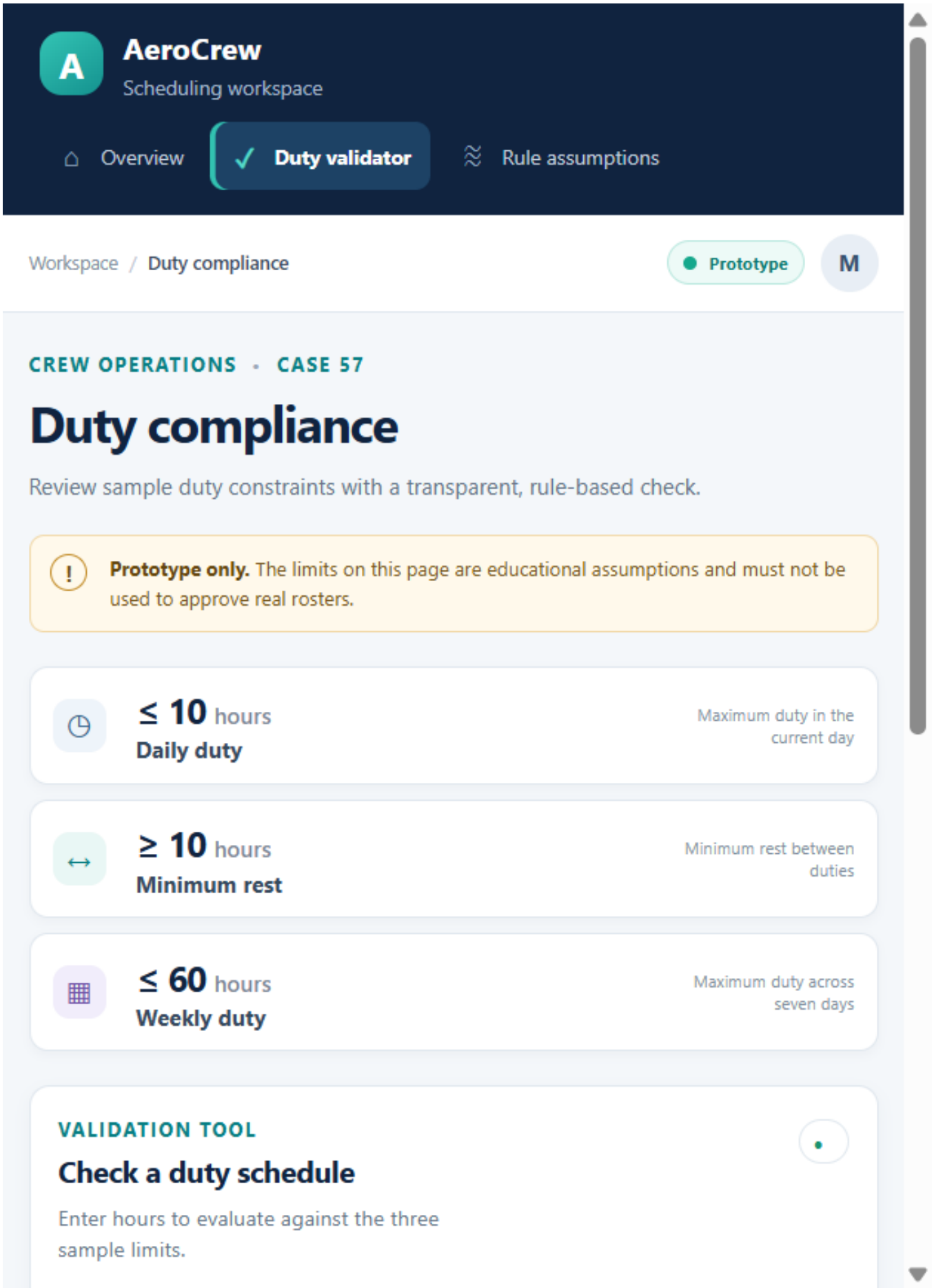


Figure 5. The same live Flask dashboard at a 390-pixel mobile viewport; the responsive navigation and rule cards are visible.

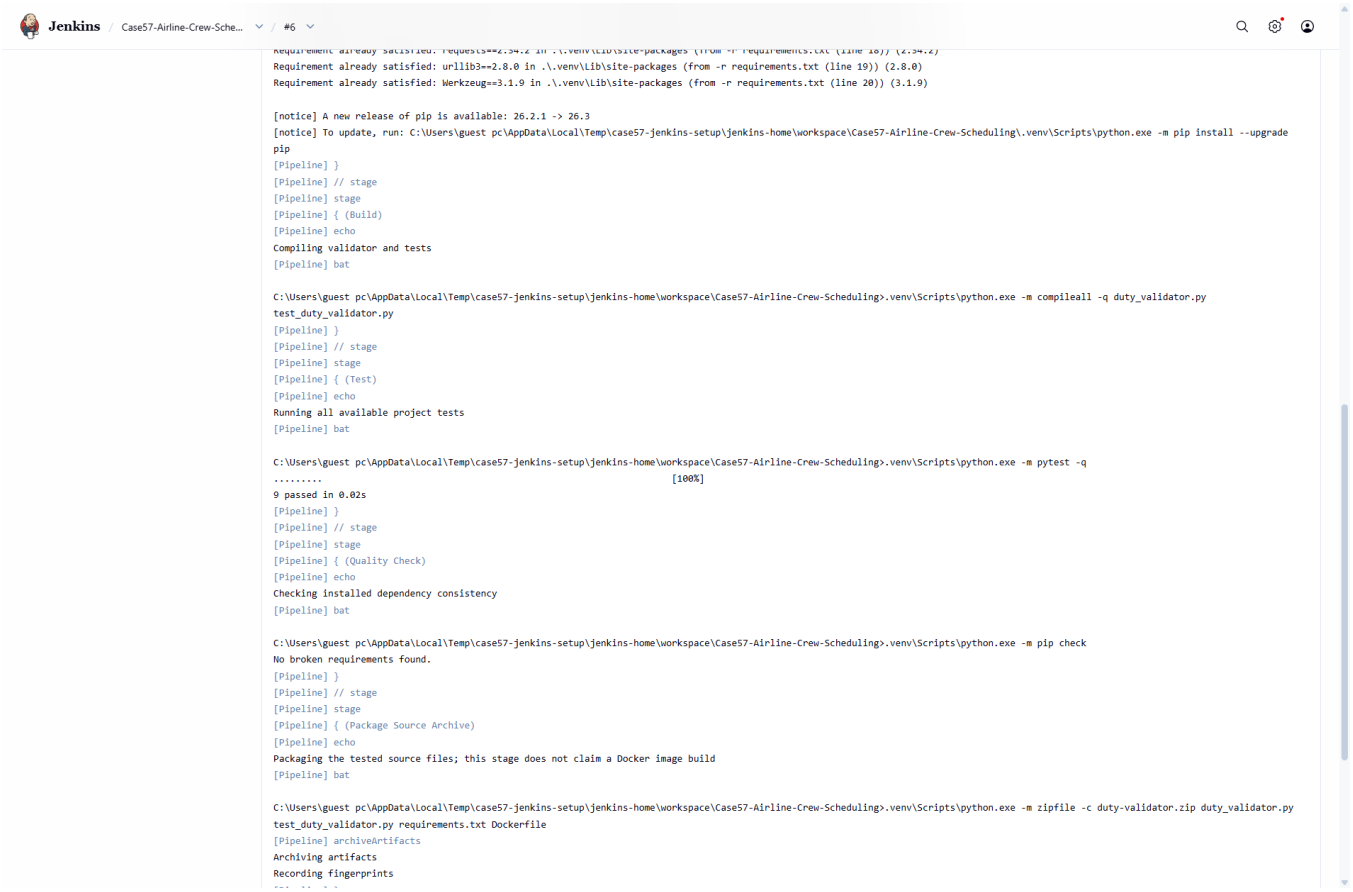


Figure 6. Genuine Jenkins build #6 console capture: pytest reports 9 passed and pip check reports no broken requirements.

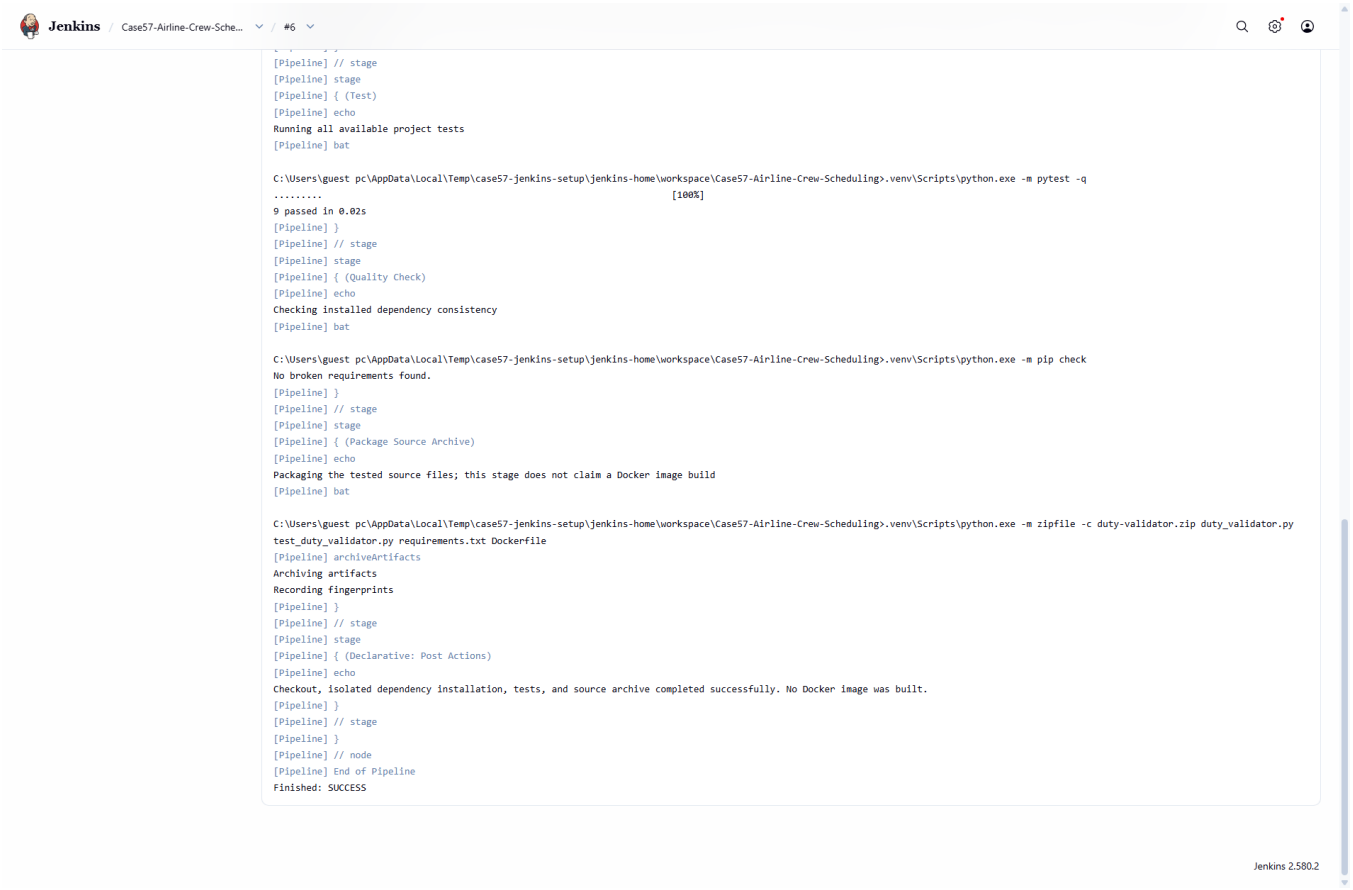


Figure 7. Genuine Jenkins build #6 console capture: source archive stage completed and final build result is SUCCESS.

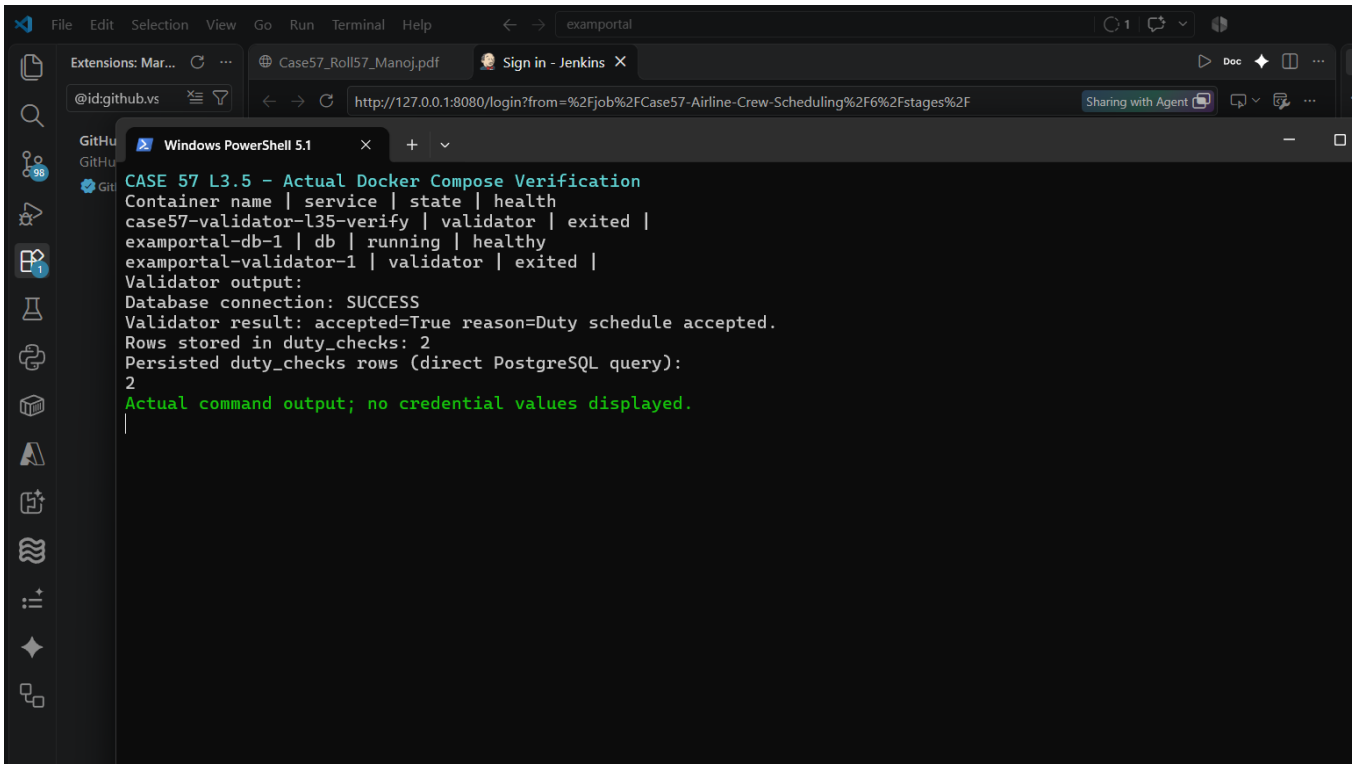


Figure 8. Genuine Compose/PostgreSQL terminal capture: one-shot validator execution and persisted-row verification.

Additional evidence inventory

Evidence	What it supports / does not support
L4.3_run_log.txt	Records Chrome/Selenium versions, both script results, and screenshot names.
UI_UX_REDESIGN/*.png	Seven new Chrome screenshots cover desktop, mobile, valid, each rule violation, reset, and input-error states.
hello.py, templates/dashboard.html, static/css/dashboard.css, static/js/validator.js	Flask route, accessible page, responsive styling, browser interaction, and shared validator API.
test_app.py	Flask HTTP/API checks for boundaries, rule reasons, missing/invalid inputs, booleans, and non-finite numbers.
JENKINS_BUILD/build_6_console.txt and genuine screenshots	Actual remote checkout, dependency install, compile, 9 passed, pip check, source archive, and final SUCCESS. The controlled failure/fix experiment is still missing.
DOCKER/compose_execution.log and compose_live_verification.png	Actual Compose build, healthy PostgreSQL, validator connection, and persisted rows; not a persistent production service.
L3.3_Evidence.md	Text transcript of isolated merge conflict/resolution; not a project-branch merge.
L5.2_Evidence.md	Three actual Pylint findings; explicitly not SonarQube.
compose.yaml, Dockerfile.compose, compose_validator_service.py	Compose configuration and one-shot validator used in the actual documented demonstration.
duty_validator.py and test files	Actual small validator, original 20-test source set, and expanded 36-test suite.

Official evidence status summary: 17 COMPLETE, 5 PARTIAL, 3 PENDING HUMAN, 0 BLOCKED. The AI-only substitutes for L2.4 (estimates), L4.2 (stakeholder feedback), and L5.1 (review perspectives) are explicitly simulated and do not change the three PENDING HUMAN statuses.